

Lista zadań nr 12

W poniższych zadaniach dotyczących rozszerzania języka z wykładu nie jest wymagane rozszerzanie kompilatora o obsługę nowych konstrukcji języka. Wystarczy rozszerzyć lekser, parser, sprawdzanie typów oraz ewaluator.

Zadanie 1. (1 pkt)

Rozszerz implementację języka z wykładu o wartość `()` oraz typ `unit`.

Zadanie 2. (2 pkt)

Rozszerz implementację języka z wykładu o typ `'a ref` oraz operacje `ref`, `!` i `(:=)`. W ewaluatorze możesz zaimplementować nowe konstrukcje używając analogicznych konstrukcji metajęzyka.

Zadanie 3. (3 pkt)

Rozszerz implementację języka z wykładu o funkcje rekurencyjne (np. definiowane przez `let rec`). Skąd wziąć typ zwracany przez funkcję rekurencyjną podczas sprawdzania/wyprowadzania typów? Może warto rozszerzyć składnię o dodatkowe informacje, podobnie jak w przypadku funkcji anonimowych?

Zadanie 4. (2 pkt)

Rozszerz język z wykładu o listy, wraz z (płytkim) dopasowaniem wzorca. Skąd wziąć typ pustej listy? Zaproponuj jakieś rozwiązanie tego problemu.

Zadanie 5. (2 pkt)

Zauważ, że implementacja kompilatora znajdująca się w SKOSie kompiluje funkcje anonimowe w środowisku `Env.frame env`, co oznacza, że funkcje te mogą odwoływać się do zmiennych z otaczającego je kontekstu tylko wtedy, gdy

zostały wprowadzone przez let-wyrażenie definiujące funkcję (`let f = fun x :  $\tau$  -> e in ...`), natomiast próba odwołania się do innej zmiennej kończy się błędem. Zmodyfikuj implementację sprawdzania typów tak, aby tego typu błędy zostały wykryte jeszcze podczas sprawdzania typów, a nie dopiero podczas kompilacji.

Zadanie 6. (2 pkt)

Przypomnij sobie algorytm unifikacji z przedmiotu *Logika dla informatyków*. Zreferuj go, a następnie uzasadnij, że algorytm ten zawsze się zatrzymuje.